

SECURITY.md

Модель безопасности платформы. Принцип: **defense in depth** — три независимых слоя (middleware → Server Action → RLS), и ни один секрет не покидает сервер.

1. Модель угроз (что защищаем)

Актив	Угроза	Последствие
Правильные ответы / решения	Ученик читает их в DevTools, через API, через параметры генерации	Обесценивание контроля, списывание
Результаты попыток	Ученик подделывает <code>is_correct</code> , время, попытки	Ложная статистика
Данные учеников (ПДн, успеваемость)	Учитель/ученик видит чужие данные, IDOR	Утечка ПДн, 152-ФЗ
Аккаунты	Брутфорс, credential stuffing, угон сессии	Компрометация
Банк заданий	Массовое скачивание (scraping)	Потеря IP
Инфраструктура	Злоупотребление генерацией/PDF (DoS, счёт за ресурсы)	Расходы, недоступность
Service-role ключ	Утечка в клиентский бандл / репозиторий	Полный доступ к БД

2. Роли и permissions

2.1 Матрица

Ресурс / действие	student	teacher	org_admin	content_editor	admin
Свой профиль: read/update	✓	✓	✓	✓	✓
Чужой профиль: read	✗	только ученики своих групп (ограниченные поля)	члены своей организации	✗	✓
Роли: change	✗	✗	в рамках org (member/teacher)	✗	✓
Группы: create/update/delete	✗	свои	в org	✗	✓
Группы: join по коду	✓	✗	✗	✗	✗
Прототипы/					

шаблоны: read published	✓	✓	✓	✓	✓
Прототипы/шаблоны: read draft, write	✗	✗	✗	✓	✓
Generated task (public view): read	свои (practice/assignment)	свои worksheet/assignment + preview	как teacher	✓	✓
answers: read	✗	только через teacher_task_answers	как teacher	✓	✓
Attempts: insert	только через submit_attempt()	✗	✗	✗	✗
Attempts: read	свои	учеников своих групп	в org	✗	✓
Assignments: write	✗	свои	в org	✗	✓
Assignments: read	адресованные мне	свои	в org	✗	✓
Worksheets/PDF	✗	свои	в org	✗	✓
Statistics: read	свои	учеников своих групп	в org (агрегаты)	✗	✓
Organizations: manage	✗	✗	своя	✗	✓
Admin panel	✗	✗	✗	КОНТЕНТ	✓

2.2 Реализация

JWT claims (Supabase Auth Hook `custom_access_token`):

```
{ "app_role": "teacher", "org_id": "...", "profile_status": "active" }
```

Хук читает `profiles` при выдаче токена. Изменение роли → принудительный `refresh` (`auth.admin.signOut(userId, 'global')`).

Хелперы в SQL (`supabase/migrations/0002_auth_helpers.sql`):

```
create function auth.role_of() returns text language sql stable
as $$ select coalesce(auth.jwt() ->> 'app_role', 'anon') $$;
create function auth.org_of() returns uuid language sql stable
as $$ select nullif(auth.jwt() ->> 'org_id', '')::uuid $$;
create function auth.is_admin() returns bool language sql stable
as $$ select auth.role_of() = 'admin' $$;
create function auth.teaches_student(sid uuid) returns bool language sql stable security definer
as $$ select exists (select 1 from group_members gm join groups g on g.id = gm.group_id
                    where gm.student_id = sid and g.teacher_id = auth.uid() and gm.left_at is null) $
```

Хелперы в коде (`src/lib/auth/guards.ts`):

```
requireUser() // сессия есть, status=active
```

```
requireRole('teacher'|...) // роль из JWT
assertGroupOwner(ctx, gid) // teacher_id = ctx.userId или org_admin той же org
assertStudentSelf(ctx, sid) // student = сам
assertCanViewStudent(ctx, sid) // сам | учитель ученика | org_admin | admin
```

Каждый Server Action обязан начинаться с guard'a. ESLint-правило кастомное: экспорт из `actions.ts` без вызова `require*` в первых строках — ошибка (проверяется в code review чек-листе, позже — lint-плагин).

3. Teacher / Student separation

- Разные route-группы (`teacher`) и (`student`); middleware редиректит по `app_role`.
- Один аккаунт = одна роль. Учитель, желающий «порешать», получает режим `preview` (задания с `origin='preview'`, не влияют на статистику).
- Учитель **не видит** попытки учеников, не входящих в его активные группы (`left_at is null`). После выхода ученика из группы — исторические данные учитель видит через агрегаты группы (`mv_*`), но не по персоне (настройка организации).
- Ученик не видит: другие группы, других учеников, учителя кроме имени и аватара.

4. Доступ к группам

RLS для `groups`:

```
create policy groups_teacher_all on groups
  for all using (teacher_id = auth.uid() or auth.is_admin()
                or (auth.role_of() = 'org_admin' and organization_id = auth.org_of()));

create policy groups_student_read on groups
  for select using (exists (select 1 from group_members gm
                           where gm.group_id = groups.id and gm.student_id = auth.uid() and gm.left_at
```

Вступление по коду — не прямой insert в `group_members`, а security definer функция `join_group(code text)`:

- проверяет `invite_enabled`, лимит участников, что вызывающий — `student`;
- rate limit 5 попыток / 10 мин на пользователя (таблица `join_attempts` или Redis);
- код 8 символов из алфавита без похожих символов (без `0/O/1/I`), энтропия ~ 40 бит; учитель может ротировать.

5. Доступ ученика только к своим данным

RLS-шаблон для всех «ученических» таблиц (`attempts`, `errors`, `statistics`, `students`, `deadlines` персональные):

```
create policy attempts_student_self on attempts
  for select using (student_id = auth.uid());
create policy attempts_teacher_read on attempts
  for select using (auth.teaches_student(student_id));
```

```
create policy attempts_org_read on attempts
  for select using (auth.role_of()='org_admin'
                    and exists (select 1 from profiles p where p.id = student_id and p.organization_id =
create policy attempts_admin on attempts
  for select using (auth.is_admin()));
-- insert/update/delete: политик нет → запрещено всем, кроме service role
```

IDOR-тест в CI: Playwright-сценарий «ученик А пытается открыть /assignments/{id_ученика_B} , /api/pdf/{чужой} » → ожидается 403/404.

6. Защита ответов и правильных ответов

6.1 Правило нулевой утечки

Правильный ответ **не существует** нигде, откуда его может прочесть клиент:

1. Таблица `answers` отделена от `generated_tasks` ; RLS не даёт `select` ни `student`, ни `teacher`.
2. `generated_tasks` для клиента отдаётся только через `view generated_tasks_public` — **без seed и без params** (по параметрам и известному шаблону ответ вычислим). Клиент получает готовый `statement_md` и SVG.
3. В клиентский бандл не попадает код `answer()` / `derive()` шаблонов: `src/tasks/templates/**` помечены `import 'server-only'` , а ESLint запрещает импорт `@/tasks/templates` из `src/components` и `src/app/**/*client.tsx` . CI-проверка: `grep` собранных `.next/static` чанков на маркер `TEMPLATE_REGISTRY` → должен отсутствовать.
4. Проверка ответа — `submitAnswer` Server Action → `service role` → `checkAnswer()` → результат (`correct: boolean` , без ожидаемого значения). Правильный ответ показывается только по политике (после N попыток / после дедлайна) через `revealSolution` , что логируется в `attempts.solution_viewed` .
5. Чертёж SVG не содержит скрытых числовых данных (проверка в тестах Geometry Engine: в SVG попадают только подписи, объявленные `visibleLabels`).
6. Режим `control` : разбор недоступен до `deadline` , повторные попытки ограничены `config.max_attempts` .

6.2 Защита результата попытки

- Клиент присылает только `given_answer` , `time_spent_ms` , `hint_used` . `is_correct` , `attempt_no` , `submitted_at` — сервер.
- `time_spent_ms` — sanity: `0 < t < 3600000` , иначе `null` + флаг.
- Идемпотентность: `idempotency_key = hash(student, task, attempt_no_expected)` ; повтор запроса возвращает тот же результат, не создаёт вторую попытку.
- Дедлайн проверяется сервером по `now()` ; поздняя попытка — по `late_policy` .

6.3 Защита банка заданий от скрапинга

- Тренировка выдаёт задания по одному; `generated_tasks_public` отдаёт только задания, на которые у ученика есть право (созданы для него или для его группы).
- Лимит генерации: 300 заданий / сутки на ученика (Free), учитель — 2000 / сутки, превью — 50.

- Preview-задания живут 24 ч (`pg_cron` чистит).

7. Rate limiting

Upstash Redis + `@upstash/ratelimit` (sliding window), ключи по `userId` или `ip` (для анонимов), реализация в `src/lib/rate-limit.ts`.

Эндпоинт / действие	Лимит
Login / register / reset (по IP)	10 / 10 мин; после 5 неудач — задержка 2 с (tarpit)
<code>join_group</code>	5 / 10 мин на пользователя
<code>submitAnswer</code>	60 / мин на ученика (человек не решает быстрее)
Генерация заданий (practice)	30 / мин, 300 / сутки
Создание worksheet / PDF	10 / час на учителя
Экспорт CSV	20 / час
Публичный API v1 (B2B)	по тарифу, per-key, 429 + <code>Retry-After</code>
Любой <code>/api/*</code> для анонима	60 / мин на IP

Supabase Auth имеет собственные лимиты (email отправка, token refresh) — используются как второй слой. Vercel WAF (Pro) — базовая защита от ботов и гео-фильтры при необходимости.

8. Базовая защита API и приложения

8.1 Транспорт и заголовки

- Только HTTPS (Vercel по умолчанию), HSTS `max-age=63072000; includeSubDomains; preload`.
- CSP (`next.config.ts` → `headers()`): `default-src 'self'; script-src 'self' 'nonce-...'; connect-src 'self' https://*.supabase.co https://*.posthog.com https://*.sentry.io; img-src 'self' data: https://*.supabase.co; frame-ancestors 'none'`.
- X-Content-Type-Options: `nosniff`, Referrer-Policy: `strict-origin-when-cross-origin`, Permissions-Policy минимальный.

8.2 Сессии

- Cookie `HttpOnly; Secure; SameSite=Lax`; refresh через `@supabase/ssr` в middleware.
- Access token TTL 1 ч, refresh — 30 дней с ротацией; «выйти везде» доступно пользователю.
- Сессия инвалидируется при смене роли/пароля/блокировке.

8.3 Валидация и мутации

- Все входы — Zod. Числа для ответов — строка → нормализация → parse; максимум 64 символа.

- Markdown учебных полей (если появятся) — санитизация (`rehype-sanitize`); формулы — KaTeX с `trust: false`.
- Server Actions защищены от CSRF Next.js (origin check); дополнительно `allowedOrigins` в `next.config`.
- Загрузка файлов: тип по magic bytes, размер ≤ 10 MB, CSV — парсинг в песочнице, без формул (`=`, `+`, `-`, `@` в начале ячейки экранируются при экспорте — защита от CSV injection).

8.4 Секреты

- `SUPABASE_SERVICE_ROLE_KEY` — только Vercel Server Env; в коде — только `src/lib/supabase/admin.ts` с `import 'server-only'`.
- Никаких `NEXT_PUBLIC_*` кроме `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`, `NEXT_PUBLIC_POSTHOG_KEY`, `NEXT_PUBLIC_SENTRY_DSN`.
- `gitleaks` в pre-commit и CI. Ротация ключей — раз в 6 месяцев и при любом подозрении.
- `.env.example` — без значений.

8.5 Зависимости

- `pnpm audit` + Dependabot еженедельно; `overrides` для уязвимых транзитивных пакетов.
- Lockfile обязателен; `--frozen-lockfile` в CI.

8.6 Мониторинг безопасности

- Sentry: алерт на всплеск 401/403 (>50/мин) — признак перебора IDOR.
- `audit_log` по чувствительным действиям; просмотр — только admin.
- Supabase Auth logs → Axiom: неудачные логины по IP.
- Раз в квартал — ручной прогон OWASP ZAP baseline против staging.

9. Персональные данные (152-ФЗ, базово)

- Минимизация: у ученика обязательны только email (или телефон позже) и имя/псевдоним.
- Согласие при регистрации сохраняется в `profiles.consent` с версией текста и датой.
- Удаление аккаунта: `anonymize_profile()` — email → `deleted+{id}@...`, имя → «Ученик», файлы — удаляются, попытки остаются обезличенными для агрегатов.
- Экспорт своих данных: `GET /api/me/export` (JSON), rate limit 1/сутки.
- Регион хранения: Supabase EU (Frankfurt) на старте; при требовании локализации — миграция на Postgres в РФ (см. `TECH_ARCHITECTURE` §1.3, вариант F) — архитектура допускает без изменения кода приложения.
- Продуктовая аналитика без ПДн (PostHog получает только `user_id`, без email/имён).

10. Чек-лист безопасности для каждого PR

- Новая таблица → RLS включён, политики написаны, есть тест.

- Новый Server Action → начинается с `require*` `guard`'а, вход через Zod.
- Нет импорта `admin.ts` / `tasks/templates` в клиентский код.
- Новый эндпоинт → `rate limit` определён.
- Нет секретов в коде и логах.
- Ошибки возвращают пользователю общий текст, детали — в Sentry.
- Если затронуты ответы/попытки — обновлён тест «ученик не может прочитать `answers`».